Seguridad de una base de datos de citas médicas

UTEC · CS2041 Bases de Datos I · Ciclo 2026-2 · Actividad grupal

Integrante A: Rojas Meza, Kiara Luz (202510620) — Actividad 1, Actividad 3 (roles y permisos) y Actividad 4 (vista y enmascaramiento)

Integrante B: Lescano Garamendi, Juan Carlos Sebastián (202510447) — Actividad 2, apoyo en la Actividad 3, Actividad 4 (riesgos) y Actividad 5

Entorno: PostgreSQL 18.4 (x86_64-windows), psql, pg_dump y pg_restore 18.4 · Base de datos `citas_medicas`

Este informe describe el diseño de una base de datos para un centro médico ficticio y las medidas de seguridad aplicadas: autenticación y conexiones cifradas, control de acceso por roles, privacidad, auditoría y respaldo. Todos los datos son inventados. Los resultados que se muestran son extractos de la salida real de PostgreSQL, incluida en la carpeta `evidencias/` de la entrega (el número entre corchetes indica el archivo, por ejemplo [E11]).

1. Diseño y creación de la base de datos

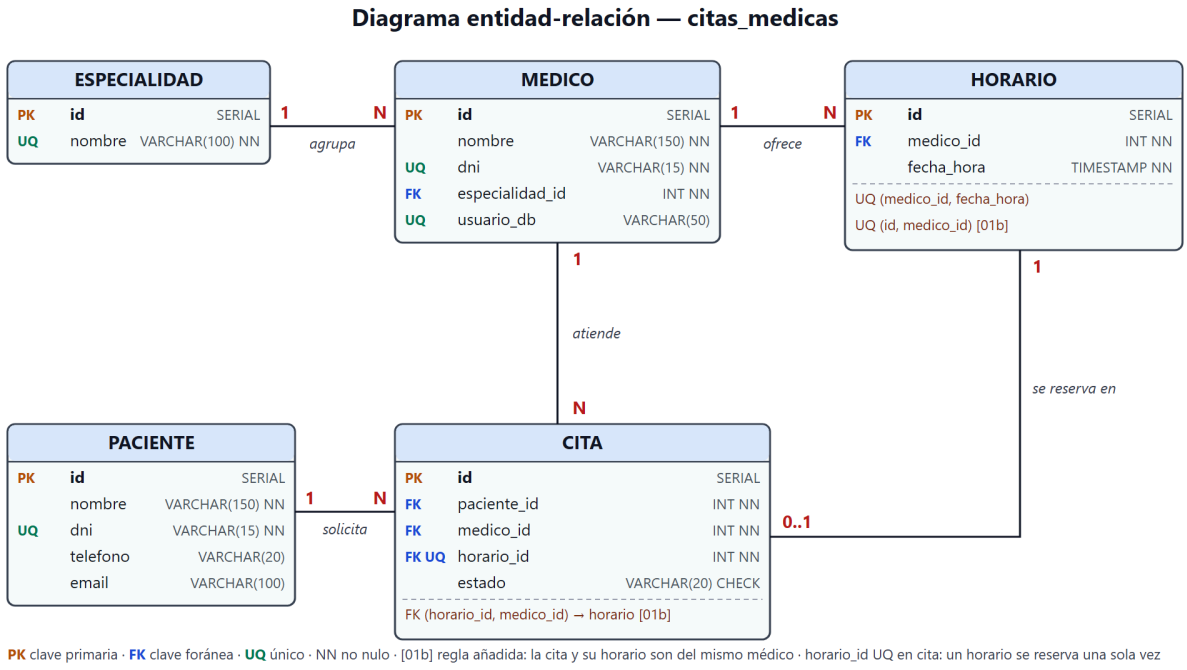


Figura 1. Diagrama entidad-relación con atributos, claves y cardinalidades.

El modelo tiene cinco entidades. Una **especialidad** agrupa a varios médicos; cada **médico** pertenece a una sola especialidad y ofrece varios **horarios**; un **paciente** puede solicitar varias **citas**, y cada cita pertenece a un paciente, a un médico y a un horario. Un horario puede reservarse como máximo una vez (relación 1 a 0..1).

Restricción	Justificación
PRIMARY KEY `id SERIAL` en todas las tablas	Identifica cada fila con un número generado automáticamente.
FOREIGN KEY con `ON DELETE RESTRICT`	No se puede borrar un paciente, un médico o un horario que tenga citas, para no perder historial.
`UNIQUE (medico_id, fecha_hora)` en horario	Un médico no puede tener dos veces el mismo horario.
`UNIQUE (horario_id)` en cita	Un mismo horario de un médico no se reserva dos veces (regla mínima del caso).
`CHECK (estado IN ('agendada','completada','cancelada'))`	Solo se admiten estados válidos.
`UNIQUE` y `NOT NULL` en dni; `UNIQUE` en `usuario_db`	Evita personas duplicadas. `usuario_db` vincula cada médico con su cuenta de PostgreSQL (lo usa la seguridad por filas).
Regla añadida (01b): `UNIQUE (id, medico_id)` en horario y FK compuesta `(horario_id, medico_id)` en cita	Con la FK simple se podía agendar una cita del médico 1 en un horario del médico 2 (se comprobó en una prueba). Con la FK compuesta, la cita y su horario siempre son del mismo médico.

Se cargaron 3 especialidades, 5 médicos, 10 pacientes, 20 horarios y 20 citas repartidas entre los cinco médicos. Las dos consultas con JOIN confirman que las relaciones funcionan [E04]:

```
-- JOIN 1: citas con paciente, médico, especialidad y horario (extracto)
 id_cita |    paciente     |       medico       | especialidad |     fecha_hora      |  estado
---------+-----------------+--------------------+--------------+---------------------+----------
       5 | Diego Sanchez   | Dra. Ana Torres    | Cardiología  | 2026-06-01 08:00:00 | agendada
       1 | Juan Perez      | Dr. Carlos Mendoza | Cardiología  | 2026-06-01 08:00:00 | agendada
       2 | Maria Lopez     | Dr. Carlos Mendoza | Cardiología  | 2026-06-01 09:00:00 | agendada
 ...
(20 filas)

-- JOIN 2: citas por especialidad
 especialidad | total_citas
--------------+-------------
 Cardiología  |           8
 Pediatría    |           8
 Dermatología |           4
(3 filas)
```

2. Tokens y conexiones seguras

2.1 Autenticación del recepcionista con JWT

```
Recepcionista            Aplicación (servidor)                          PostgreSQL
  | 1. usuario + contraseña (HTTPS) -->|                                      |
  |                                    | 2. verifica la contraseña (hash)     |
  |<-- 3. JWT firmado (sub=recep01,    |                                      |
  |       rol=recepcion, exp=15 min)   |                                      |
  | 4. solicitud + "Bearer JWT" ------>| 5. valida firma, vencimiento y rol   |
  |                                    | 6. consulta con SU PROPIA cuenta de BD ---->| 7. aplica GRANT,
  |                                    |    (miembro de recepcionista, por TLS)    |    RLS y vistas
  |<------------------- 8. respuesta --|<------------------------------------------|
```

La recepcionista inicia sesión en la aplicación, no en PostgreSQL. La aplicación comprueba su contraseña y le entrega un **JWT**: un token firmado con una clave que solo conoce la aplicación, con su identidad, su rol y un vencimiento corto. En las solicitudes siguientes la recepcionista envía el token y la aplicación verifica la firma antes de atenderla. **El JWT no es la contraseña de PostgreSQL**: la base de datos nunca lo recibe. La aplicación se conecta con una cuenta propia, miembro del rol `recepcionista`, que solo puede hacer lo que permite la matriz de la sección 3. Así, aunque alguien robara un token, no obtendría más permisos que esos y solo por unos minutos.

2.2 Verificación de SSL/TLS

Consulta [E08]	Resultado	Interpretación
`SHOW ssl;`	`off`	El servidor no tiene TLS habilitado (no hay certificado ni clave configurados).
`SELECT ssl FROM pg_stat_ssl WHERE pid = pg_backend_pid();`	`f`	La sesión actual no está cifrada; `version` y `cipher` salen vacíos.
Reglas de `pg_hba.conf`	solo `host`	No hay reglas `hostssl`: el servidor no exige TLS a ningún cliente.
Conexión con `sslmode=require`	rechazada	*«el servidor no soporta SSL, pero SSL es requerida»*.
Conexión con `sslmode=prefer` (por defecto)	`ssl = f`	El cliente intenta TLS y, al no estar disponible, continúa sin cifrar y sin avisar.

¿Por qué `SHOW ssl = on` no garantiza el cifrado? Ese valor solo indica que el servidor *puede* aceptar TLS. Que cada conexión vaya cifrada depende además de `pg_hba.conf`, donde una regla `host` acepta conexiones con o sin TLS y solo `hostssl` lo exige, y del `sslmode` del cliente: con `disable` o `prefer` puede quedar sin cifrar, `require` cifra sin verificar el certificado y `verify-full` cifra y comprueba la identidad del servidor. Por eso el cifrado se verifica sesión por sesión con `pg_stat_ssl`.

> **Limitación:** el servidor de prueba es una instalación local con la configuración por defecto y no ofrece TLS, por lo que no se pudo mostrar una sesión cifrada (no se simuló ningún resultado). La contraseña viaja protegida por SCRAM-SHA-256, pero los datos de las consultas irían en claro si la conexión saliera del equipo; aquí cliente y servidor están en la misma máquina (`::1`). En producción habría que instalar un certificado, activar `ssl = on`, cambiar las reglas `host` por `hostssl` y usar `sslmode=verify-full` en la aplicación.

3. Control de acceso

Se crearon tres **roles de grupo** sin inicio de sesión (`NOLOGIN`) y tres **cuentas de prueba** con `LOGIN`, miembros de cada grupo. Las contraseñas no están en los scripts: cada integrante las define en su equipo con `\password`. Matriz aplicada [E07]:

Rol (cuenta)	Permisos concedidos	Justificación
administrador (`admin_user`)	SELECT, INSERT, UPDATE y DELETE en todas las tablas; política RLS `FOR ALL`; `ALTER DEFAULT PRIVILEGES` de lectura para los objetos creados después (vistas y auditoría).	Administra el sistema. No recibe TRUNCATE ni TRIGGER porque no los necesita. Sobre la auditoría solo puede leer.
recepcionista (`recep_user`)	SELECT en `vista_recepcion`; INSERT y UPDATE en `cita`; SELECT solo de la columna `cita.id`; SELECT en `horario`; USAGE en `cita_id_seq`; políticas RLS de SELECT, INSERT y UPDATE.	Permisos adicionales justificados: `cita.id` lo exige el `WHERE id = X` del UPDATE, y `horario` (sin datos personales) es necesario para encontrar un horario libre. Sin permisos sobre `paciente`.
médico (`medico_carlos`)	SELECT en `cita` filtrado por RLS; SELECT de `medico.id` y `medico.usuario_db`; SELECT en `vista_agenda_medico`.	Solo lectura de sus propias citas. La política necesita leer esas dos columnas para encontrar su id. No tiene acceso a `paciente`.

Restricción por cuenta (RLS). La tabla `cita` tiene seguridad a nivel de filas. La política del médico es `USING (medico_id = (SELECT id FROM medico WHERE usuario_db = CURRENT_USER))`, de modo que PostgreSQL agrega ese filtro aunque la consulta no lo tenga. La vista `vista_agenda_medico` muestra además el nombre del paciente; como su dueño es `postgres` y no pasa por la RLS, lleva su propio filtro `WHERE m.usuario_db = CURRENT_USER` y la opción `security_barrier`.

Ensayos. Cada ensayo se hizo **iniciando sesión directamente** con la cuenta de prueba (`psql -U <cuenta>`). En todos, `SELECT current_user, session_user` devolvió esa misma cuenta.

N.º	Cuenta	Sentencia	Resultado esperado	Resultado obtenido
1	recep_user	`SELECT * FROM vista_recepcion ORDER BY id_cita;`	Permitido: datos para gestionar citas, sin documento.	20 filas con paciente, teléfono, médico, especialidad, fecha y estado [E09].
2	recep_user	`UPDATE cita SET estado = 'completada' WHERE id = 1;` y luego consulta a la vista	Permitido: el cambio queda guardado.	`UPDATE 1`; la vista muestra la cita 1 pasando de *agendada* a *completada* [E10].
3	medico_carlos	`SELECT id, paciente_id, medico_id, horario_id, estado FROM cita;` (sin WHERE) y `SELECT * FROM vista_agenda_medico;`	Permitido: solo sus citas (médico 1), aunque hay citas de 5 médicos.	4 filas (citas 1 a 4), todas con `medico_id = 1`; `count(DISTINCT medico_id) = 1`. La vista devuelve las mismas 4 citas [E11].
4	recep_user	`SELECT dni FROM paciente;`	Denegado por permisos.	ERROR: permiso denegado a la tabla paciente [E12]
5	medico_carlos	`UPDATE cita SET estado = 'cancelada' WHERE id = 2;`	Denegado por permisos.	ERROR: permiso denegado a la tabla cita [E13]

La primera versión del script de roles se revisó antes de estos ensayos: el médico recibía *permiso denegado a la tabla medico*, el administrador veía 0 citas por no tener política RLS y la recepción no podía hacer el UPDATE por id. Estos problemas se corrigieron en la versión final de `03_roles.sql`.

4. Privacidad e ISO/IEC 27001:2022

Vista de recepción. `vista_recepcion` une cita, paciente, médico, especialidad y horario, y muestra solo lo necesario para gestionar citas: nombre y teléfono del paciente, médico, especialidad, fecha y estado. Excluye el **documento (dni)** y el correo. La recepción puede consultar la vista (ensayo 1), pero PostgreSQL le niega la lectura directa del documento (ensayo 4).

Enmascaramiento. El script `04_vistas.sql` muestra el valor original junto a su versión protegida [E06]:

```
 id |     nombre      | dni_original | dni_enmascarado |     email_original      | email_enmascarado
----+-----------------+--------------+-----------------+-------------------------+-------------------
  1 | Juan Perez      | 70000001     | ****0001        | juan.perez@email.com    | ju***@email.com
  2 | Maria Lopez     | 70000002     | ****0002        | maria.lopez@email.com   | ma***@email.com
  3 | Pedro Ramirez   | 70000003     | ****0003        | pedro.ramirez@email.com | pe***@email.com
 ...
(10 filas)
```

El enmascaramiento sirve para mostrar un dato a quien solo necesita reconocerlo, por ejemplo confirmar los últimos dígitos del documento por teléfono, sin exponer el valor completo, y para usar datos en reportes o pruebas con menos riesgo. No convierte

los datos en anónimos: junto con el nombre, el paciente sigue siendo identificable.

Tabla de riesgos según ISO/IEC 27001:2022

Riesgo	Descripción	Impacto	Mitigación aplicada	Vínculo con ISO/IEC 27001
R1. Lectura no autorizada	Una cuenta con más permisos de los necesarios, o una conexión sin cifrar, deja ver el documento o el teléfono de los pacientes, o las citas de otros médicos.	Exposición de datos personales y de salud; pérdida de confianza; incumplimiento de la normativa de protección de datos.	Roles con mínimo privilegio (GRANT y REVOKE); vista sin documento; RLS para que cada médico vea solo sus citas; enmascaramiento; recomendación de TLS con `hostssl` y `verify-full`.	Se identifica y valora en la evaluación de riesgos (cláusula 6.1.2) y se reduce en el tratamiento (6.1.3) con controles del Anexo A: control de acceso (5.15), restricción del acceso a la información (8.3), enmascaramiento de datos (8.11) y criptografía (8.24).
R2. Pérdida de registros	Un borrado accidental, una falla del disco o un error al ejecutar scripts elimina pacientes, horarios o citas.	No se puede atender a los pacientes agendados; se pierde el historial y se interrumpe la atención.	Respaldo con `pg_dump -Fc` y restauración probada en una base vacía, con conteos y JOIN idénticos; claves foráneas con `ON DELETE RESTRICT`.	Riesgo de disponibilidad e integridad (6.1.2) tratado con el control de copias de seguridad (8.13). La prueba de restauración verifica que el control funciona (evaluación del desempeño, cláusula 9).
R3. Cambios sin trazabilidad	Una cita se crea, cambia o elimina sin que quede registro de quién lo hizo ni cuándo.	No se pueden investigar errores o abusos ni responder a un reclamo.	Trigger de auditoría sobre `cita` con operación, fecha y hora, cuenta y datos antes y después; tabla legible solo por el administrador.	Riesgo de integridad y de responsabilidad (6.1.2) tratado con el control de registro de eventos (8.15) y la protección de esos registros.

En los tres casos se siguió el ciclo de la norma: **identificar** el riesgo, **valorar** su impacto, elegir un **tratamiento** (aquí, reducirlo con controles técnicos) y **verificar** con pruebas registradas que el control funciona.

5. Auditoría y respaldo

5.1 Auditoría con trigger

`05_auditoria.sql` crea la tabla `auditoria_cita` (`id`, `operacion`, `fecha_hora DEFAULT now()`, `usuario_bd`, `cita_id`, `datos_anteriores` y `datos_nuevos` en jsonb) y un trigger `AFTER INSERT OR UPDATE OR DELETE ... FOR EACH ROW` sobre `cita`. La función es `SECURITY DEFINER` con `SET search_path = public`: así la recepción no necesita permiso para escribir en la auditoría, y nadie puede desviar la escritura a otra tabla con el mismo nombre. El id afectado se toma con `COALESCE(NEW.id, OLD.id)`. Solo el administrador puede leer la tabla.

Usuario registrado. Se guarda `SESSION_USER`, es decir, la **cuenta de PostgreSQL** con la que se inició sesión. No se usa `CURRENT_USER` porque, dentro de una función `SECURITY DEFINER`, devuelve al dueño de la función (`postgres`). El registro **no** identifica al usuario de la aplicación: si todos entraran por la aplicación de la sección 2, todas las filas mostrarían su cuenta técnica. Para guardarlo también, la aplicación podría ejecutar `SET LOCAL app.usuario = 'recep01'` al inicio de cada transacción (con el usuario del JWT ya validado), y el trigger lo leería con `current_setting('app.usuario', true)` en una columna adicional. Ese valor lo informa la aplicación y PostgreSQL no puede verificarlo, por eso conviene conservar ambos datos.

Prueba. `admin_user` creó un horario libre [E15]. Luego `recep_user` registró una cita en ese horario y repitió la modificación del ensayo 2 [E16]. Por último, `admin_user` eliminó la cita de prueba y consultó el historial [E17]:

```
 id | operacion |          fecha_hora           | usuario_bd | cita_id | estado_antes | estado_despues
----+-----------+-------------------------------+------------+---------+--------------+----------------
  1 | INSERT    | 2026-11-10 22:54:56.945237-05 | recep_user |      21 |              | agendada
  2 | UPDATE    | 2026-11-10 22:54:56.964161-05 | recep_user |       1 | completada   | completada
  3 | DELETE    | 2026-11-10 22:54:57.045514-05 | admin_user |      21 | agendada     |
(3 filas)
```

Cada operación quedó registrada con la cuenta que la hizo, la fecha y hora y el id de la cita. En el UPDATE el estado figura igual antes y después porque se repitió exactamente la modificación del ensayo 2, que ya había dejado la cita en *completada*. Aun así, el trigger registró la operación, porque la fila fue actualizada.

5.2 Respaldo y restauración

```
pg_dump -Fc -h localhost -U postgres -d citas_medicas -f citas_medicas.backup      (código 0, 27 750 bytes)
createdb -h localhost -U postgres citas_restaurada                                  (código 0)
pg_restore -h localhost -U postgres -d citas_restaurada citas_medicas.backup        (código 0, sin errores)
```

Tabla [E20]	citas_medicas	citas_restaurada	¿Coincide?
especialidad	3	3	Sí
medico	5	5	Sí
paciente	10	10	Sí
horario	21	21	Sí
cita	20	20	Sí
auditoria_cita	3	3	Sí

Hay 21 horarios porque se conserva el horario libre que creó el administrador para la prueba de auditoría. La consulta JOIN 1 se ejecutó en ambas bases (20 filas cada una) y `diff` no encontró diferencias [E21]. El respaldo incluye tablas, datos, restricciones (también la FK compuesta), vistas, función, trigger, políticas RLS y permisos.

> **Los roles no viajan en el respaldo.** `pg_dump` guarda una sola base y los roles pertenecen a todo el servidor: el listado del respaldo no tiene ningún `CREATE ROLE` [E18]. La restauración funcionó porque se hizo en el mismo servidor. En otro servidor hay que crear antes los roles (ejecutando `03_roles.sql` o con `pg_dumpall --roles-only`) o restaurar con `--no-owner --no-privileges`. Además, antes de volver a ejecutar `03_roles.sql` hay que borrar `citas_restaurada`, porque contiene permisos de esos roles y `DROP ROLE` fallaría.

Referencias

Ramakrishnan, R. y Gehrke, J. *Sistemas de gestión de bases de datos* (3.ª ed.). McGraw-Hill. Temas consultados: diseño conceptual con el modelo entidad-relación, y seguridad y autorización.

The PostgreSQL Global Development Group. *PostgreSQL 18 Documentation*: Database Roles; Privileges; Row Security Policies; Secure TCP/IP Connections with SSL; The pg_hba.conf File; SSL Support (sslmode) en libpq; Triggers; pg_dump; pg_restore. https://www.postgresql.org/docs/18/

ISO/IEC 27001:2022. *Seguridad de la información, ciberseguridad y protección de la privacidad — Sistemas de gestión de la seguridad de la información — Requisitos*. ISO/IEC.

Jones, M., Bradley, J. y Sakimura, N. (2015). *JSON Web Token (JWT)*. RFC 7519, IETF. https://www.rfc-editor.org/rfc/rfc7519